JUNIOR DEVOPS ENGINEER EXAM 2026

PART 5 | SOLUTION PRESENTATION

From foundations

to reliable delivery.

A presentation of what I built, what the evidence shows, and what I learned.

THE STORY

Foundation
Packaging
Automation
Availability
Reflection

PRESENTED BY

Mark Andrei Castillo

Linux | Docker | CI/CD | Kubernetes HA

OVERALL APPROACH

I worked from the foundation upward

01 | FOUNDATION

Linux commands
Permissions
Logs and processes
Bash scripts

02 | PACKAGING

FastAPI API
Next.js UI
MySQL dependency
Docker Compose

03 | DELIVERY

GitHub Actions
Build and test
Docker Hub tags
Notifications

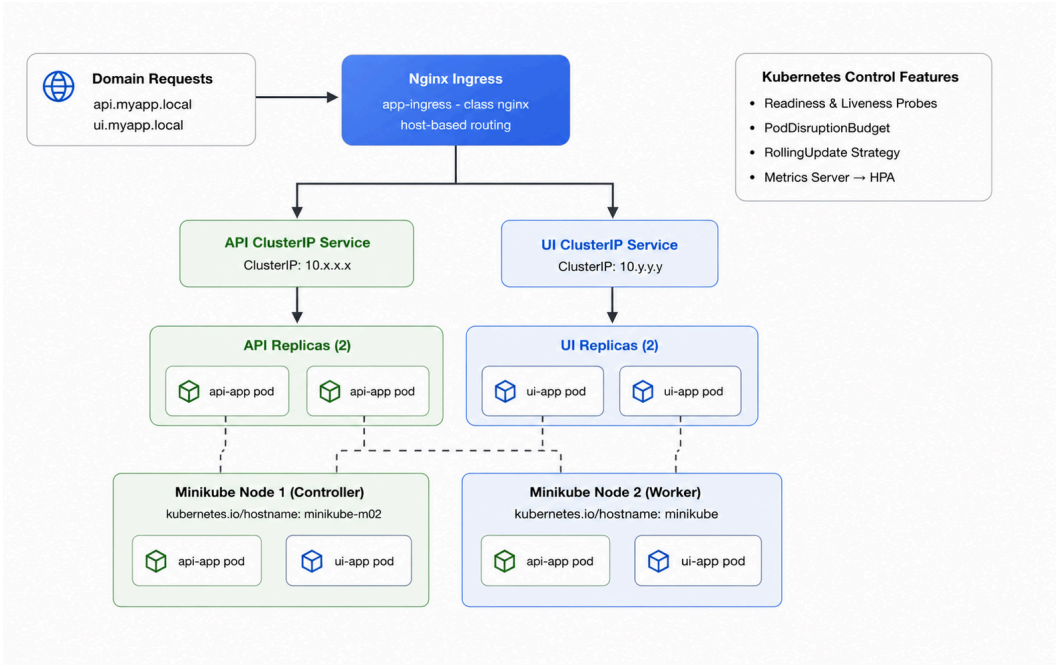
04 | RESILIENCE

Kubernetes
Two nodes
Replicas and probes
Failover evidence

ARCHITECTURE

One application path, from local development to HA runtime

APPLICATION AND DEPLOYMENT ARCHITECTURE



REPOSITORY ARCHITECTURE EVIDENCE

LOCAL

Compose connects the API, UI, and MySQL for repeatable development.

DELIVERY

GitHub Actions verifies the change before publishing traceable container tags.

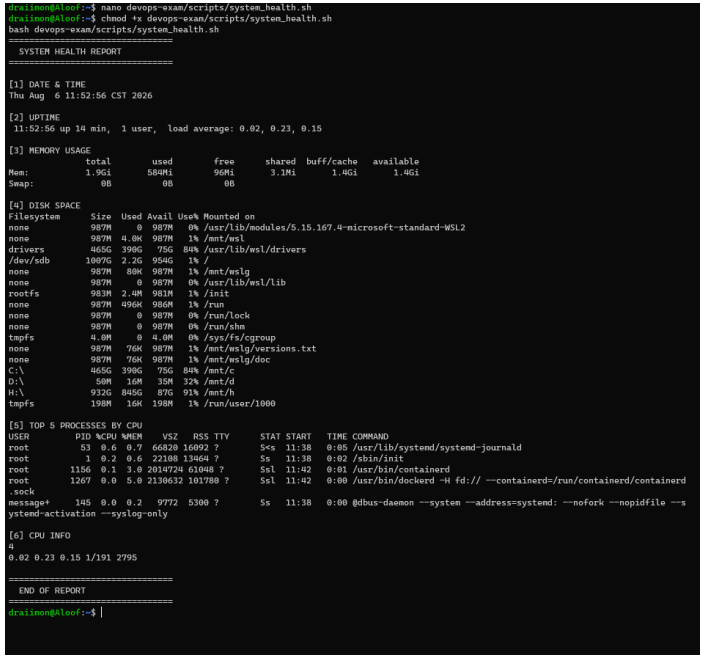
RUNTIME

Kubernetes adds replicas, Services, Ingress, health checks, scaling, and disruption protection.

PART 1 | LINUX BASICS

I made the machine state visible before building on top of it

SYSTEM HEALTH AND OPERATIONAL CHECKS



LOCAL WSL CAPTURE

WHAT I COMPLETED

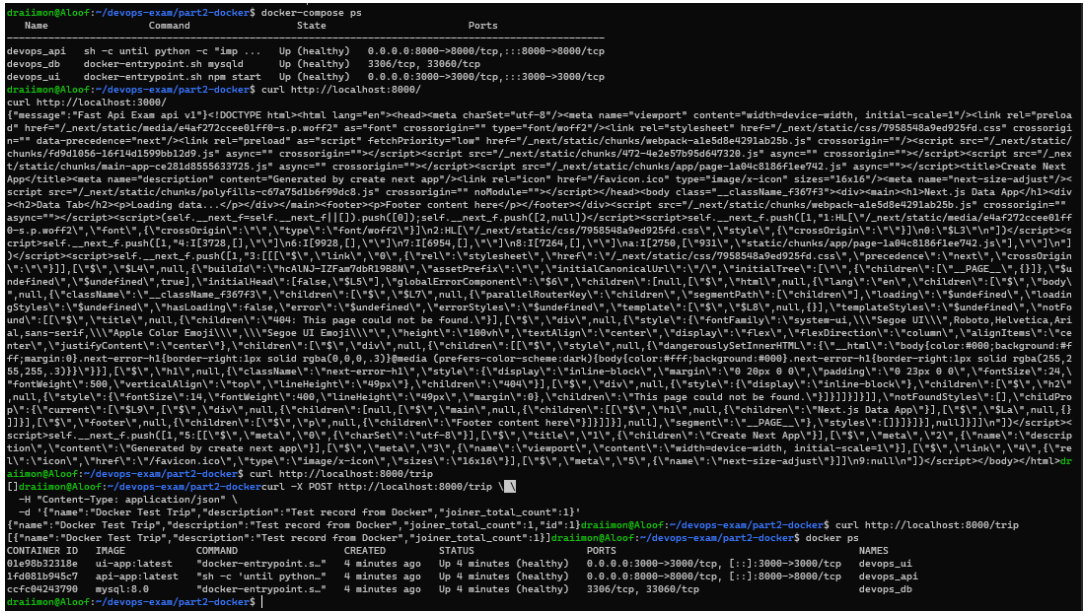
File operations, permissions, searching, processes, networking, packages, users, archives, and Bash scripts.

WHAT I LEARNED

Repeatable checks are more useful than isolated commands. The same habit helped later with Docker and Kubernetes troubleshooting.

PART 2 | DOCKER CONTAINERIZATION

API, UI, AND DATABASE RUNNING TOGETHER



LOCAL WSL CAPTURE

API

FastAPI, Python slim, non-root user, health check.

UI

Next.js, Node Alpine, production build, non-root user.

WHY THIS DESIGN

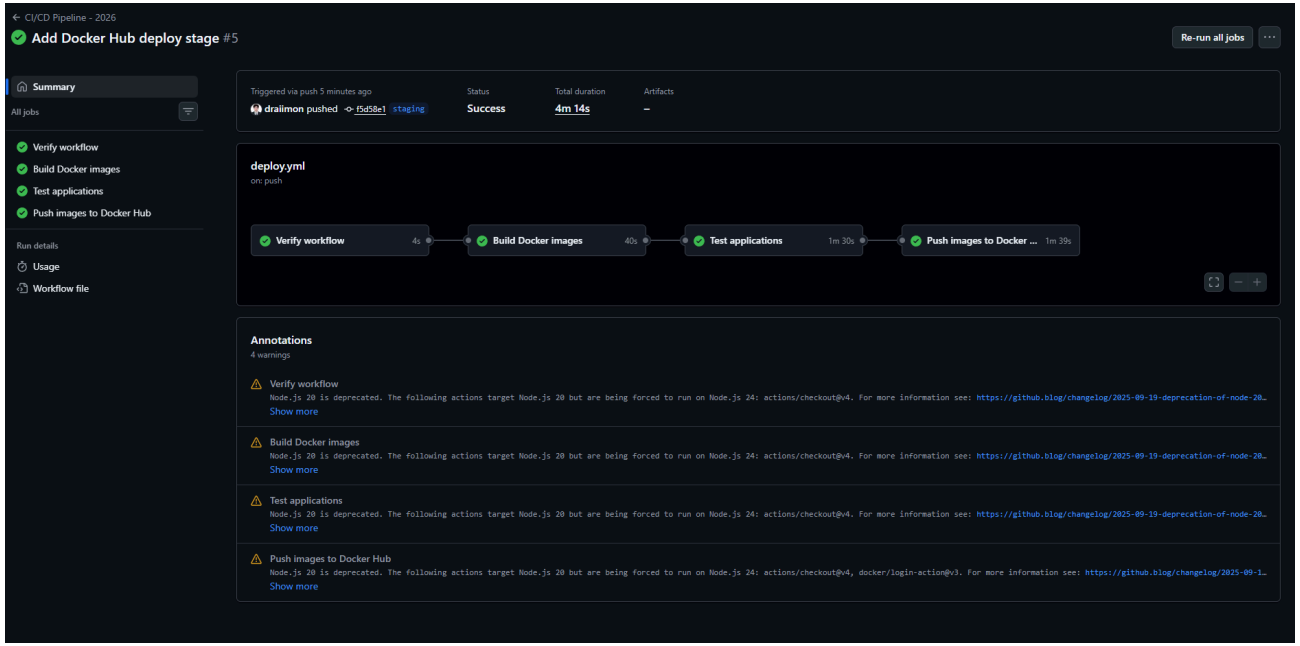
Separate runtimes make the services easier to build, test, and scale independently.

Result: the API, UI, and MySQL services were running and healthy before automation.

PART 3 | CI/CD PIPELINE

The pipeline is a quality gate, not only a deployment button

SUCCESSFUL GITHUB ACTIONS PIPELINE



CHECKPOINTED CI EVIDENCE

TRIGGER

Push to staging starts the workflow.

QUALITY

Verify, build, and test run before publishing.

TRACEABILITY

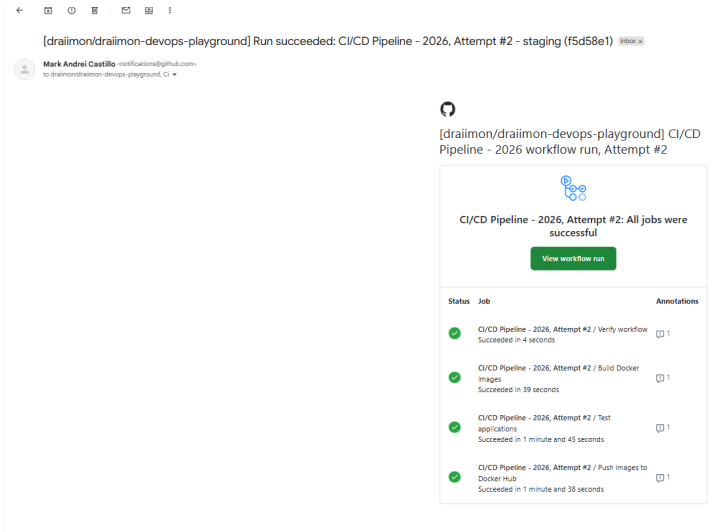
SHA, staging, and latest tags identify the result.

RESULT Build, test, publish, notify: passed.

PART 3 | EVIDENCE

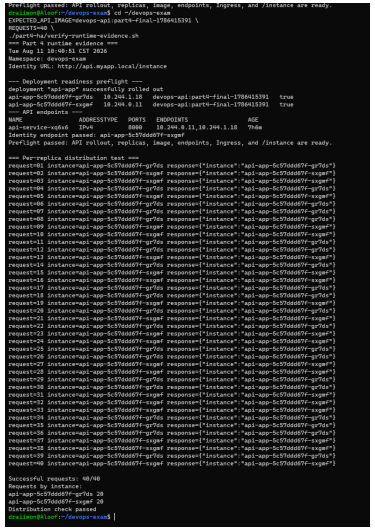
The captured result shows the delivery path completed

SUCCESS NOTIFICATION EVIDENCE



CHECKPOINTED EVIDENCE

LATER RUNTIME EVIDENCE



PART 4 RUNTIME EVIDENCE

HONEST EVIDENCE BOUNDARY

PART 4 | HIGH AVAILABILITY DESIGN

I chose Kubernetes because the controls match the requirement

REDUNDANCY

Two API replicas
Two UI replicas
Two Minikube nodes
Topology spread
RollingUpdate

TRAFFIC

ClusterIP Services
Ingress host routing
Ready endpoints
Local domain access
Stable service path

RECOVERY

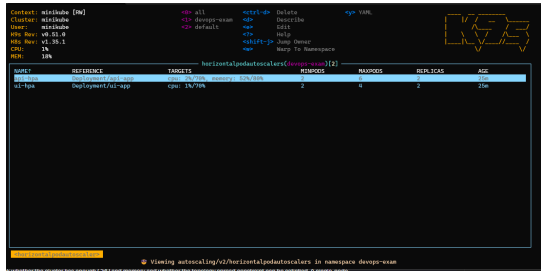
Readiness and liveness
HPA scaling
PodDisruptionBudget
Node-failure test
Automatic replacement

Trade-off: Minikube is practical and reproducible for the exam, but a managed cluster would be the production next step.

PART 4 | RUNTIME EVIDENCE

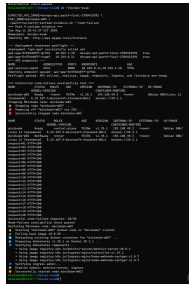
The runtime checks the story the manifests promised

HPA METRICS AND REPLICA STATE



LOCAL WSL / K9S CAPTURE

NODE FAILURE: 20/20 HTTP 200



LOCAL WSL CAPTURE

WHAT THE EVIDENCE SHOWS

The deployment was not treated as complete because the YAML existed.

The captured checks show:
- ready replicas and expected images
- Service endpoint membership
- domain-based routing
- traffic reaching both replicas
- HTTP 200 responses while one node was unavailable

CHALLENGES AND DECISIONS

The hardest problems were boundary problems

SOURCE AND IMAGE

Challenge: early containers did not contain the real application source.
Resolution: clone the source and rebuild from the correct context.

DATABASE ALIGNMENT

Challenge: the initial database setup did not match the API.
Resolution: align the Compose stack with MySQL.

FRESH CI RUNNERS

Challenge: one job cannot rely on state from another job.
Resolution: each job declares its own setup and readiness checks.

RUNTIME TRUTH

Challenge: rollout success does not guarantee traffic health.
Resolution: preflight images, endpoints, routing, and real requests.

Decision principle: make the boundary observable before trusting the next layer.

TRADE-OFFS AND NEXT IMPROVEMENTS

I can explain what is complete and what I would improve

WHY THESE TOOLS

Docker Compose: simple local feedback loop.

GitHub Actions: repeatable checks on a fresh runner.

Kubernetes: replicas, routing, health, scaling, and recovery in one model.

Minikube: realistic Kubernetes behavior without cloud setup.

WITH MORE TIME

Add vulnerability scanning as a required policy gate.

Deploy with immutable image digests.

Run and document a real rollback test.

Move from local Minikube to managed staging.

Add metrics, logs, and alerting beyond screenshots.

PART 5 | CLOSING

Reliable delivery is a chain of verified boundaries

Linux made the system observable.

Docker made services portable.

CI/CD made delivery repeatable.

Kubernetes made availability testable.

Questions and discussion.